

Trabajo final Sistemas de Aprendizaje Automático

Superordenador Urederra

Javier Peralta



Índice

1- Estructura de carpetas	3
2- Modelo 2 CPUs .py	3
3- Modelo 2 CPUs .sbatch	6
4a- Registro de real, user y sys por CPU	7
4b- Registro de loss_train, acc_train y acc_test por CPU	7
5- (capturas)	8
5- (Registro de datos por GPU)	11

1- Estructura de carpetas

```
[u0080011@urlnprohpc1gn02 trabajo71]$ ls -l
total 16
drwxr-sr-x 2 u0080011 ug_008_cursoiabigdata_usr 4096 May 18 17:41 datasets71
drwxr-sr-x 2 u0080011 ug_008_cursoiabigdata_usr 4096 May 18 17:39 logs71
drwxr-sr-x 2 u0080011 ug_008_cursoiabigdata_usr 4096 May 19 19:56 src71
drwxr-sr-x 6 u0080011 ug_008_cursoiabigdata_usr 4096 May 18 17:50 venv-ml71

[u0080011@urlnprohpc1gn02 trabajo71]$ ls -l datasets71/
total 179776
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 184089106 May 18 16:58 TotalFeatures-ISCXFlowMeter.csv

[u0080011@urlnprohpc1gn02 trabajo71]$ ls -l src71/
total 84
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 7075 May 19 18:41 red_neuronal_urederra_cpu16_71.py
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 698 May 19 18:41 red_neuronal_urederra_cpu16_71.sbatch
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 7074 May 19 18:27 red_neuronal_urederra_cpu1_71.py
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 694 May 19 17:44 red_neuronal_urederra_cpu1_71.sbatch
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 7074 May 19 18:31 red_neuronal_urederra_cpu2_71.py
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 694 May 19 17:46 red_neuronal_urederra_cpu2_71.sbatch
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 7074 May 19 18:39 red_neuronal_urederra_cpu4_71.py
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 694 May 19 18:36 red_neuronal_urederra_cpu4_71.sbatch
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 7074 May 19 18:40 red_neuronal_urederra_cpu8_71.py
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 695 May 19 18:40 red_neuronal_urederra_cpu8_71.sbatch
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 7219 May 19 19:09 red_neuronal_urederra_gpu1_71.py
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 797 May 19 17:42 red_neuronal_urederra_gpu1_71.sbatch
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 7143 May 19 19:16 red_neuronal_urederra_gpu2_71.py
-rw-r--r-- 1 u0080011 ug_008_cursoiabigdata_usr 797 May 19 19:12 red_neuronal_urederra_gpu2_71.sbatch
```

2- Modelo 2 CPUs .py

Al ejecutar en urederra, me decía que me daba nulos en y al convertir a 0 y 1. He ejecutado con unos prints para debugear y he encontrado que hay un valor que es “ge”, por eso en el mapping añadido como valor 1 a “ge”.

```
CPUs asignadas por Slurm: 1
NaN en y: 1
Valores unicos originales (tras limpiar):
class
benign          471596
adware          155613
generalmalware  4745
ge              1
Name: count, dtype: int64
Unmapped: <StringArray>
['ge']
```

```

Sistemas de aprendizaje automático > Trabajo Final > red_neuronal_urederra_cpu2_71.py > ...
1 import os
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 import pandas as pd
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8
9 # Número de hilos CPU que utilizará PyTorch
10 num_cpus = int(os.environ.get("SLURM_CPUS_PER_TASK", 2))
11 torch.set_num_threads(num_cpus)
12
13 # Mostramos el número de hilos activos
14 print("CPUs asignadas por Slurm:", num_cpus)
15
16 # Lectura del archivo csv y conversión a un dataframe
17 df = pd.read_csv('/data/scratch/008/IA_BigData/trabajo71/datasets71/TotalFeatures-ISCXFlowMeter.csv', delimiter = ',')
18
19 # Conversión de datos: de pandas a tensores en PyTorch
20 X = df.drop(columns=['class'])
21 y = df['class']
22 y = y.astype(str).str.strip().str.lower()
23
24 # Convertimos las clases categóricas a números
25 y = y.map({
26     'benign': 0,
27     'adware': 1,
28     'generalmalware': 1,
29     'ge': 1
30 })
31
32 # Convertimos los valores anteriores a Numpy
33 X_np = X.to_numpy()
34 y_np = y.to_numpy()
35
36 # transformamos los datos de Numpy a tensores de PyTorch
37 X_tensor = torch.tensor(X_np, dtype=torch.float32)
38 y_tensor = torch.tensor(y_np, dtype=torch.float32)
39
40 # Para nuestro modelo (salida 1 neurona) y debe tener forma (n, 1)
41 y_tensor = y_tensor.view(-1, 1) # también reshape(-1,1)
42
43 # Definimos el modelo de red neuronal
44 modelo = nn.Sequential(
45
46     # CAPA 1
47     # 79 entradas y 128 neuronas
48     nn.Linear(X.shape[1], 128, bias=True, dtype=torch.float32),
49
50     # Función de activación ReLU
51     nn.ReLU(),
52
53     # CAPA 2
54     # 128 entradas y 64 neuronas
55     nn.Linear(128, 64, bias=True, dtype=torch.float32),
56
57     # Función de activación ReLU
58     nn.ReLU(),
59
60     # CAPA 3
61     # 64 entradas y 32 neuronas
62     nn.Linear(64, 32, bias=True, dtype=torch.float32),
63
64     # Función de activación ReLU
65     nn.ReLU(),
66
67     # CAPA DE SALIDA
68     # 32 entradas y 1 neurona
69     nn.Linear(32, 1, bias=True, dtype=torch.float32),
70
71     # Sigmoid para clasificación binaria
72     nn.Sigmoid()
73 )
74
75 # definimos la función de pérdida
76 loss_fn = nn.BCELoss()
77
78 # definimos el optimizador
79 optimizador = optim.Adam(modelo.parameters(), lr = 0.001)
80
81 # Dividir en entrenamiento y prueba
82 X_train, X_test, y_train, y_test = train_test_split(
83     X_tensor, # Matriz de características de entrada (features)-atributos de entrada-variables predictoras
84     y_tensor, # Vector/serie con la variable objetivo (labels-en este caso: outcome)
85     test_size=0.2, # Proporción para el conjunto de prueba y bel conjunto de entrenamiento: 20% test, 80% train
86     random_state=42, # Semilla para que la división sea reproducible
87     shuffle=True, # Barajar aleatoriamente antes de dividir (recomendado si no hay una serie temporal)
88     stratify=y_tensor, # garantiza que la distribución de clases de la variable objetivo se mantiene en los conjuntos de entrenamiento y prueba

```

```

Sistemas de aprendizaje automático > Trabajo Final > red_neuronal_urederra_cpu2_71.py > ...
92 )
93
94
95 # Instanciamos un objeto StandardScaler
96 scaler_std = StandardScaler()
97
98 # Ajustamos el escalador SÓLO con los datos de entrenamiento
99 X_train_scaled = scaler_std.fit_transform(X_train)
100
101 # Aplicar la transformación al conjunto de prueba (sin volver a ajustar)
102 X_test_scaled = scaler_std.transform(X_test)
103
104 # Convertimos arrays de numpy a tensor.
105 X_train = torch.tensor(X_train_scaled, dtype=torch.float32)
106 X_test = torch.tensor(X_test_scaled, dtype=torch.float32)
107
108 # inicio del entrenamiento
109 # número de recorridos del dataset completo
110 n_epocas = 50
111
112 # número de filas del dataset que componen un batch
113 tamanho_batch = 64
114
115 # número de batches (enteros) que hay en una época. Ignora los datos sobrantes
116 batches_por_epoca = len(X_train) // tamanho_batch
117
118 # listas para guardar la evolución de las métricas
119 lista_loss_train = []
120 lista_acc_train = []
121 lista_acc_test = []
122
123 # Repite todo el entrenamiento n_epocas veces
124 for epoca in range(n_epocas):
125
126     # MODELO EN MODO ENTRENAMIENTO
127     modelo.train()
128
129     # acumuladores para calcular métricas medias
130     loss_acumulada = 0.0
131     acc_acumulada = 0.0
132
133     # Recorremos todos los batches de esta época. i es el número de batch
134     for i in range(batches_por_epoca):
135
136         # Calculamos el índice inicial del batch actual
137         # Si i = 0, empieza en 0
138         # Si i = 1, empieza en 10
139         # Si i = 2, empieza en 20
140
141         inicio_batch = i * tamanho_batch
142
143         # Calculamos el índice final del batch actual
144         fin_batch = inicio_batch + tamanho_batch
145
146         # Seleccionamos un subconjunto de datos (batch)
147         X_batch = X_train[inicio_batch : fin_batch ]
148
149         # Seleccionamos las etiquetas reales correspondientes al batch
150         y_batch = y_train[inicio_batch : fin_batch]
151
152         # El modelo hace una predicción (forward propagation).
153         # En la primera pasada actualiza aleatoriamente los parámetros del modelo.
154         # Los pesos solo se actualizan después de loss.backward() y optimizador.step()
155         y_pred = modelo(X_batch)
156
157         # Calculamos el error entre predicción y valor real
158         loss = loss_fn(y_pred, y_batch)
159
160         # PyTorch acumula gradientes, no los reemplaza. Borra gradientes anteriores
161         # Pone a cero (borra) en este batch los gradientes acumulados en todos los parámetros del modelo
162         # En PyTorch los gradientes no se reemplazan, se acumulan. Quiero SOLO el gradiente de este batch
163         optimizador.zero_grad()
164
165         # Backward propagation: Cálculo del gradiente. Matemáticamente: dL/dW
166         # calculamos automáticamente las derivadas de la pérdida respecto a los parámetros del modelo
167         loss.backward()
168
169         # Actualizamos los pesos del modelo W = W - learning_rate * gradiente(derivadas)
170         optimizador.step()
171
172         acc = (y_pred.round() == y_batch).float().mean()
173
174         loss_acumulada += loss.item()
175         acc_acumulada += acc.item()
176
177     # métricas medias de entrenamiento de la época
178     loss_media = loss_acumulada / batches_por_epoca
179     acc_media = acc_acumulada / batches_por_epoca
180
181     # MODELO EN MODO EVALUACIÓN
182
183     # Cambia el modelo a modo evaluación (inferencia) con el conjunto de test. Durante el entrenamiento, el modelo aprende ajustando sus pesos.
184     # Ahora ya no queremos aprender, solo queremos hacer predicciones sobre X_test, compararlas con las etiquetas reales y_test
185     # y calcular la exactitud (accuracy)
186     modelo.eval()
187
188     # No calculamos gradientes, más rápido y menos memoria.

```

```

188     # Ejecuta el modelo sin calcular ni almacenar gradientes, solo se quiere predecir
189     with torch.no_grad():
190         y_pred_test = modelo(X_test)
191         acc_test = (y_pred_test.round() == y_test).float().mean()
192
193     # Guardamos métricas de la época
194     lista_loss_train.append(loss_media)
195     lista_acc_train.append([acc_media])
196     lista_acc_test.append(acc_test.item())
197
198     # mostramos métricas del modelo para el conjunto de test
199     print(
200         f"Época {epoca}: "
201         f"loss_train={loss_media:.4f}, "
202         f"acc_train={acc_media:.4f}, "
203         f"acc_test={acc_test:.4f}"
204     )

```

3- Modelo 2 CPUs .sbatch

Al ejecutar el sbatch y leer el archivo de salida me salía un error que decía que me estaban haciendo kill al proceso.

```

[u0080011@urlnprohpc1gn02 src71]$ cat ../logs71/red_neuronal_84746.out
=====
Trabajo PyTorch multicore CPU
JOB ID: 84746
CPUs asignadas: 1
Nodo: urlnprohpcn143
=====
/var/spool/slurmd/job84746/slurm_script: line 24: 2372116 Killed                  python red_neuronal_urederra_cpu1_71.py
real 12.77
user 9.98
sys 1.98
slurmstepd-urlnprohpcn143: error: Detected 1 oom kill event in StepId=84746.batch. Some of the step tasks have been OOM Killed.

```

Así que dupliqué la cantidad de RAM por cada CPU (4G, 8G...)

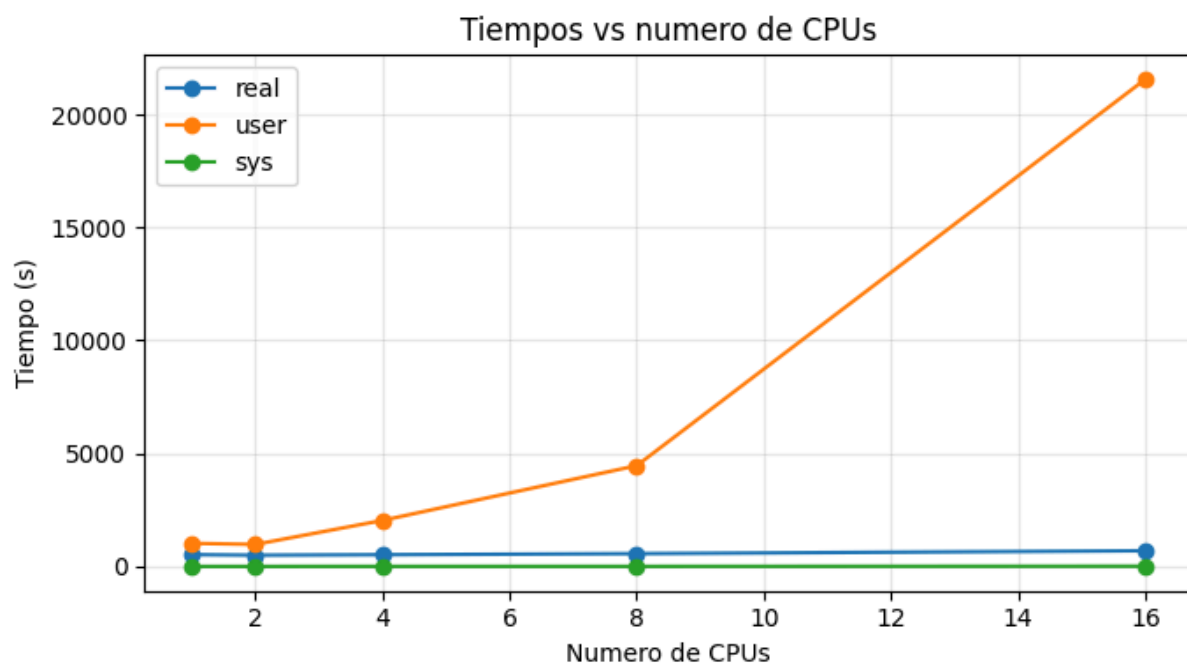
```

Sistemas de aprendizaje automático > Trabajo Final > $ red_neuronal_urederra_cpu2_71.sbatch
1  #!/bin/bash
2
3  #SBATCH --job-name=red_neuronal_cpu2
4  #SBATCH --partition=intel_skylake
5  #SBATCH --cpus-per-task=2
6  #SBATCH --mem=8G
7  #SBATCH --time=01:10:00
8  #SBATCH --output=/data/scratch/008/IA_BigData/trabajo71/logs71/red_neuronal_%j.out
9  #SBATCH --chdir=/data/scratch/008/IA_BigData/trabajo71/src71
10
11  module purge
12
13  module load Python/3.12.3-GCCcore-13.3.0
14
15  source /data/scratch/008/IA_BigData/trabajo71/venv-ml71/bin/activate
16
17  echo "=====
18  echo "Trabajo PyTorch multicore CPU"
19  echo "JOB ID: $SLURM_JOB_ID"
20  echo "CPUs asignadas: $SLURM_CPUS_PER_TASK"
21  echo "Nodo: $SLURMD_NODENAME"
22  echo "=====
23
24  time -p python red_neuronal_urederra_cpu2_71.py

```

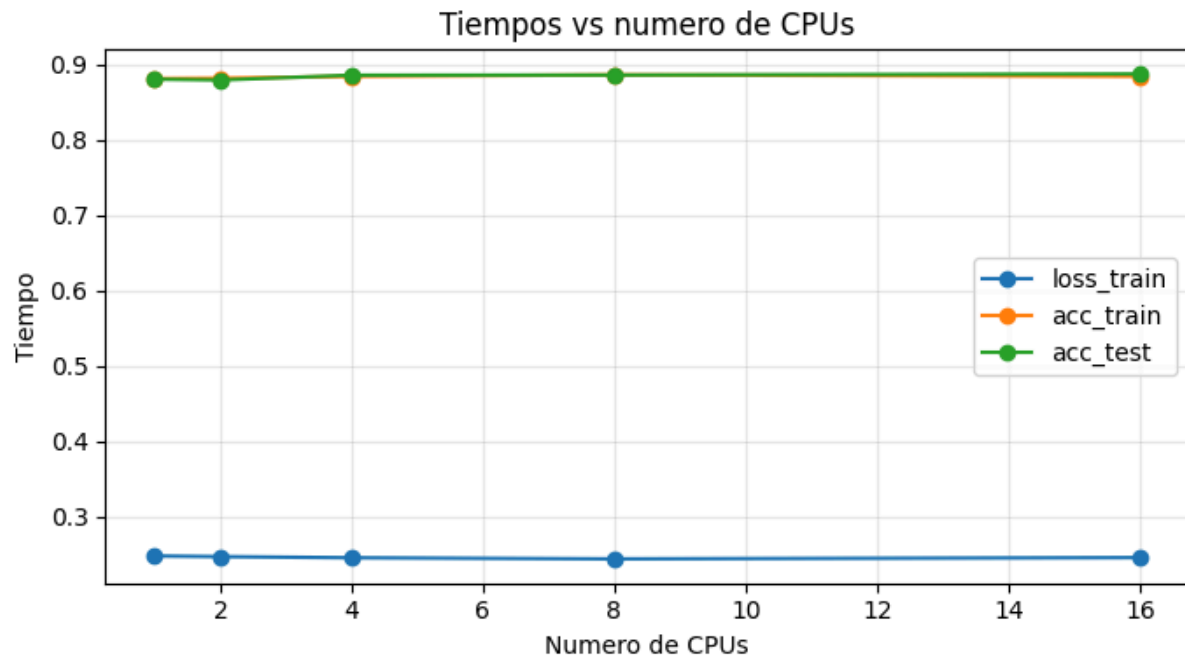
4a- Registro de real, user y sys por CPU

CPUs	real	user	sys
1	528.44	1029.00	8.17
2	504.90	981.89	8.40
4	525.45	2042.66	10.03
8	573.11	4465.47	10.41
16	699.77	21543.49	17.12



4b- Registro de loss_train, acc_train y acc_test por CPU

CPUs	loss_train	acc_train	acc_test
1	0.2475	0.8814	0.8806
2	0.2466	0.8819	0.8795
4	0.2451	0.8843	0.8859
8	0.2436	0.8863	0.8861
16	0.2453	0.8844	0.8876



5- (capturas)

Archivo

.py

```

Sistemas de aprendizaje automático > Trabajo Final > red_neuronal_urederra_gpu2_71.py > ...
1  import os
2  import torch
3  import torch.nn as nn
4  import torch.optim as optim
5  import pandas as pd
6  from sklearn.model_selection import train_test_split
7  from sklearn.preprocessing import StandardScaler
8
9  # GPUs
10 device = torch.device("cuda")
11
12 print("Nombre GPU:", torch.cuda.get_device_name(0))
13
14 # Lectura del archivo csv y conversión a un dataframe
15 df = pd.read_csv('/data/scratch/008/IA_BigData/trabajo71/datasets71/TotalFeatures-ISCXFlowMeter.csv', delimiter = ',')
16
17 # Conversión de datos: de pandas a tensores en PyTorch
18 X = df.drop(columns=['class'])
19 y = df['class']
20 y = y.astype(str).str.strip().str.lower()
21
22 # Convertimos las clases categóricas a números
23 y = y.map({
24     'benign': 0,
25     'adware': 1,
26     'generalmalware': 1,
27     'ge': 1
28 })
29
30 # Convertimos los valores anteriores a Numpy
31 X_np = X.to_numpy()
32 y_np = y.to_numpy()
33
34 # transformamos los datos de Numpy a tensores de PyTorch
35 X_tensor = torch.tensor(X_np, dtype=torch.float32)
36 y_tensor = torch.tensor(y_np, dtype=torch.float32)
37
38 # Para nuestro modelo (salida 1 neurona) y debe tener forma (n, 1)
39 y_tensor = y_tensor.view(-1, 1) # también reshape(-1,1)
40
41 # Definimos el modelo de red neuronal
42 modelo = nn.Sequential(

```


Sistemas de aprendizaje automático > Trabajo Final > red_neuronal_urederra_gpu2_71.py > ...

```
45     # 79 entradas y 128 neuronas
46     nn.Linear(X.shape[1], 128, bias=True, dtype=torch.float32),
47
48     # Función de activación ReLU
49     nn.ReLU(),
50
51
52     # CAPA 2
53     # 128 entradas y 64 neuronas
54     nn.Linear(128, 64, bias=True, dtype=torch.float32),
55
56     # Función de activación ReLU
57     nn.ReLU(),
58
59
60     # CAPA 3
61     # 64 entradas y 32 neuronas
62     nn.Linear(64, 32, bias=True, dtype=torch.float32),
63
64     # Función de activación ReLU
65     nn.ReLU(),
66
67
68     # CAPA DE SALIDA
69     # 32 entradas y 1 neurona
70     nn.Linear(32, 1, bias=True, dtype=torch.float32),
71
72     # Sigmoid para clasificación binaria
73     nn.Sigmoid()
74 )
75
76 # Movemos el modelo completo a GPU
77 modelo = modelo.to(device)
78
79 # definimos la función de pérdida
80 loss_fn = nn.BCELoss()
81 # definimos el optimizador
82 optimizador = optim.Adam(modelo.parameters(), lr = 0.001)
83
84
85 # Dividir en entrenamiento y prueba
86 X_train, X_test, y_train, y_test = train_test_split(
```

```

Sistemas de aprendizaje automático > Trabajo Final > red_neuronal_urederra_gpu2.71.py > ...
85 X_tensor, # Matriz de características de entrada (features)-atributos de entrada-variables predictoras
86 y_tensor, # Vector/serie con la variable objetivo (labels-en este caso: outcome)
87 test_size=0.2, # Proporción para el conjunto de prueba y el conjunto de entrenamiento: 20% test, 80% train
88 random_state=42, # Semilla para que la división sea reproducible
89 shuffle=True, # Barajar aleatoriamente antes de dividir (recomendado si no hay una serie temporal)
90 stratify=y_tensor # garantiza que la distribución de clases de la variable objetivo se mantiene en los conjuntos de entrenam
91 )
92
93
94 # Instanciamos un objeto StandardScaler
95 scaler_std = StandardScaler()
96
97 # Ajustamos el escalador SÓLO con los datos de entrenamiento
98 X_train_scaled = scaler_std.fit_transform(X_train)
99
100 # Aplicar la transformación al conjunto de prueba (sin volver a ajustar)
101 X_test_scaled = scaler_std.transform(X_test)
102
103 # Convertimos arrays de numpy a tensor y los movemos a GPU
104 X_train = torch.tensor(X_train_scaled, dtype=torch.float32).to(device)
105 X_test = torch.tensor(X_test_scaled, dtype=torch.float32).to(device)
106
107 # Movemos también las etiquetas a GPU
108 y_train = y_train.to(device)
109 y_test = y_test.to(device)
110
111 # Inicio del entrenamiento
112 # número de recorridos del dataset completo
113 n_epocas = 50
114
115 # número de filas del dataset que componen un batch
116 tamanho_batch = 64
117
118 # número de batches (enteros) que hay en una época. Ignora los datos sobrantes
119 batches_por_epoca = len(X_train) // tamanho_batch
120
121 # Listas para guardar la evolución de las métricas
122 lista_loss_train = []
123 lista_acc_train = []
124 lista_acc_test = []
125
126 # Repite todo el entrenamiento n_epocas veces

```

```

for epoca in range(n_epocas):

    # MODELO EN MODO ENTRENAMIENTO
    modelo.train()

    # acumuladores para calcular métricas medias
    loss_acumulada = 0.0
    acc_acumulada = 0.0

    # Recorremos todos los batches de esta época. i es el número de batch
    for i in range(batches_por_epoca):

        # Calculamos el índice inicial del batch actual
        # Si i = 0, empieza en 0
        # Si i = 1, empieza en 10
        # Si i = 2, empieza en 20
        inicio_batch = i * tamanho_batch

        # Calculamos el índice final del batch actual
        fin_batch = inicio_batch + tamanho_batch

        # Seleccionamos un subconjunto de datos (batch)
        X_batch = X_train[inicio_batch : fin_batch ]

        # Seleccionamos las etiquetas reales correspondientes al batch
        y_batch = y_train[inicio_batch : fin_batch]

        # El modelo hace una predicción (forward propagation).
        # En la primera pasada actualiza aleatoriamente los parámetros del modelo.
        # Los pesos solo se actualizan después de loss.backward() y optimizador.step()
        y_pred = modelo(X_batch)

        # Calculamos el error entre predicción y valor real
        loss = loss_fn(y_pred, y_batch)

        # PyTorch acumula gradientes, no los reemplaza. Borra gradientes anteriores
        # Pone a cero (borra) en este batch los gradientes acumulados en todos los parámetros del modelo
        # En PyTorch los gradientes no se reemplazan, se acumulan. Quiero SOLO el gradiente de este batch
        optimizador.zero_grad()

        # Backward propagation: Cálculo del gradiente. Matemáticamente: dL/dW
        # calculamos automáticamente las derivadas de la pérdida respecto a los parámetros del modelo

```

```
169     loss.backward()
170
171     # Actualizamos los pesos del modelo  $W = W - \text{learning\_rate} * \text{gradiente}(\text{derivadas})$ 
172     optimizador.step()
173
174     acc = (y_pred.round() == y_batch).float().mean()
175
176     loss_acumulada += loss.item()
177     acc_acumulada += acc.item()
178
179     # métricas medias de entrenamiento de la época
180     loss_media = loss_acumulada / batches_por_epoca
181     acc_media = acc_acumulada / batches_por_epoca
182
183     # MODELO EN MODO EVALUACIÓN
184
185     # Cambia el modelo a modo evaluación (inferencia) con el conjunto de test. Durante el entrenamiento, el modelo aprende ajustando su
186     # Ahora ya no queremos aprender, solo queremos hacer predicciones sobre  $X_{\text{test}}$ , compararlas con las etiquetas reales  $y_{\text{test}}$ 
187     # y calcular la exactitud (accuracy)
188     modelo.eval()
189
190     # No calculamos gradientes, más rápido y menos memoria.
191     # Ejecuta el modelo sin calcular ni almacenar gradientes, solo se quiere predecir
192     with torch.no_grad():
193         y_pred_test = modelo(X_test)
194         acc_test = (y_pred_test.round() == y_test).float().mean()
195
196     # Guardamos métricas de la época
197     lista_loss_train.append(loss_media)
198     lista_acc_train.append(acc_media)
199     lista_acc_test.append(acc_test.item())
200
201     # mostramos métricas del modelo para el conjunto de test
202     print(
203         f"Época {epoca}: "
204         f"loss_train={loss_media:.4f}, "
205         f"acc_train={acc_media:.4f}, "
206         f"acc_test={acc_test:.4f}"
207     )
```

Archivo .sbatch

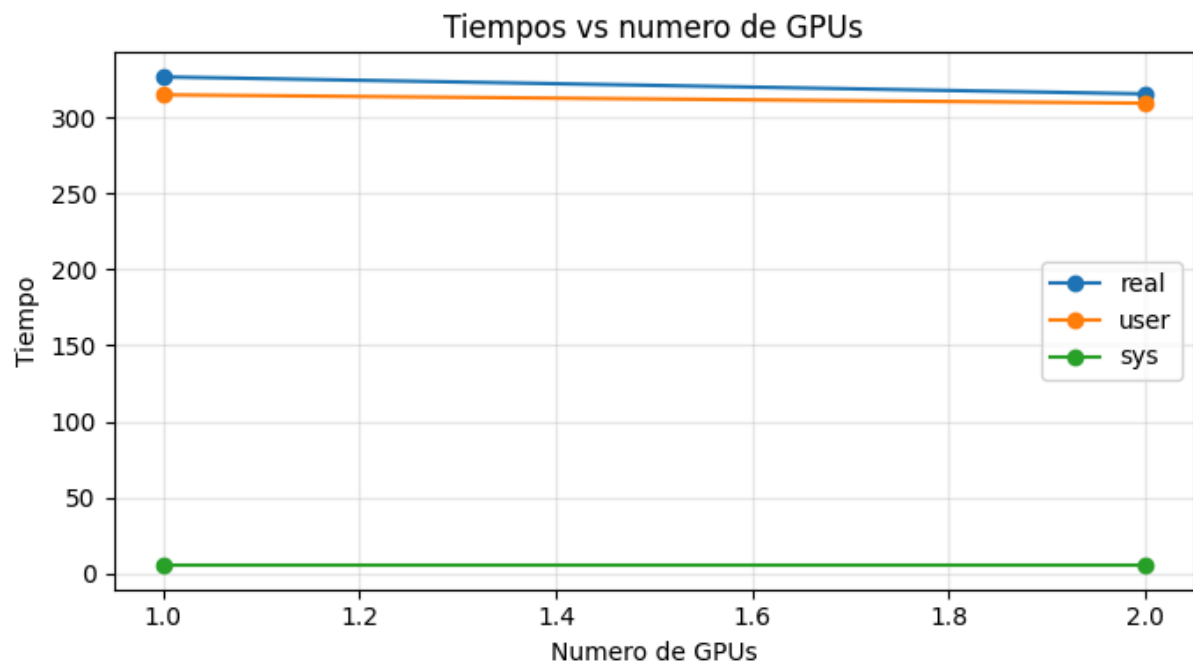
```

Sistemas de aprendizaje automático > Trabajo Final > $ red_neuronal_urederra_gpu2_71.sbatch
1  #!/bin/bash
2
3  #SBATCH --job-name=red_neuronal_gpu2
4  #SBATCH --partition=nvidia_h100
5  #SBATCH --gres=gpu:2
6  #SBATCH --cpus-per-task=12
7  #SBATCH --mem=32G
8  #SBATCH --time=00:10:00
9  #SBATCH --output=/data/scratch/008/IA_BigData/trabajo71/logs71/red_neuronal_%j.out
10 #SBATCH --chdir=/data/scratch/008/IA_BigData/trabajo71/src71
11
12 module purge
13
14 module load Python/3.12.3-GCCcore-13.3.0
15
16 source /data/scratch/008/IA_BigData/trabajo71/venv-ml71/bin/activate
17
18 echo "=====
19 echo "Trabajo PyTorch GPU"
20 echo "JOB ID: $SLURM_JOB_ID"
21 echo "CPUs asignadas: $SLURM_CPUS_PER_TASK"
22 echo "Nodo: $SLURMD_NODENAME"
23 echo "GPU asignada: $CUDA_VISIBLE_DEVICES"
24 echo "=====
25
26 # NVIDIA System Management Interface
27 nvidia-smi
28
29 time -p python red_neuronal_urederra_gpu2_71.py

```

5- (Registro de datos por GPU)

GPUs	real	user	sys
1	326.67	314.90	5.61
2	315.50	309.38	5.61



GPUs	loss_train	acc_train	acc_test
1	0.2433	0.8858	0.8830
2	0.2457	0.8832	0.8865

